

RS256: The disadvantage of HS256 is that we need the secret key both while encoding and decoding the token. Thus it needs to be present on each server that is handling the authentication. This makes the key less secure.

In RS256, we use a public private key pair as a secret key. A private key is used to sign the token and the public key is used to verify the signature of the token. This private key is kept for the server that has generated it while the public key can be shared with anyone in the world. Anyone having this public key can verify the validity of the token.

The following code shows how to use the RS256 encoding algorithm:

```
from flask import Flask, session, url_for, redirect
import jwt
import datetime
import time

app = Flask(__name__)
app.secret_key = 'super secret key'
base_url = "http://127.0.0.1:5000"
private_key_name = 'jwt_key'
public_key_name = 'jwt_key.pub'

@app.route('/')
@app.route('/<result>')
def home(result=None):
    return f"""<html>
        <body>
            <center>
                <h2>Welcome</h2>
                <hr>
                <a href="{base_url}/generate">Get
token</a><br><br>
                <a href="{base_url}/validate">Show
details</a>
                <p>Result - {result}</p>
            </center>
        </body>
    </html>"""

@app.route('/generate')
def generate_token():
    try:
        payload = {
            'sub': 'bob',
            'iat': datetime.datetime.utcnow(),
            'exp': datetime.datetime.utcnow() +
datetime.timedelta(days=0,seconds=60)
        }

        with open(private_key_name) as f:
```

```
private_key = f.read()

        token = jwt.encode(payload, private_key,
algorithm='RS256')
        session['jwt_token']=token
    except Exception as e:
        return redirect(url_for('home', result="Error 400"))

        return redirect(url_for('home', result="Token Generated
successfully!!"))

@app.route('/validate')
def validate_and_show():
    try:
        token = session.get('jwt_token')
        with open(public_key_name) as f:
            public_key = f.read()
            decode = jwt.decode(token, public_
key,algorithms=["RS256"])
            return redirect(url_for('home',
result=decode['sub']))
    except:
        return redirect(url_for('home', result="Invalid
token"))

if __name__ == '__main__':
    app.run(debug=True)
```

This code is similar to the code explained in HS256. A few changes that I have made are as follows.

1. First, a new public private key pair using the *ssh-keygen* command is generated. Do read the tutorial at <https://www.ssh.com/academy/ssh/keygen> to learn about *ssh-keygen* commands.
2. In the generate_token API, the private key file is read and passed to the encode method as a secret key. Also, the algorithm name is RS256.
3. In the validate_and_show API, the public key of the same pair is read and passed to the decode method as a secret key. The *algorithms* list will contain 'RS256' in it.

This application works the same way as described in the case of HS256. But in comparison to HS256, the secret key is more secure.

I hope this article has given you the information you need to get started with implementing JWT in your application. **END** 🐧

 By: Anchal Agarwal

The author is a software enthusiast who actively builds and contributes to open source.